

# Le Vrai Travail Caché — Docker & RAG

Pour Sahar : ce qui se passe vraiment derrière les exemples simples

---

## PARTIE 1 — DOCKER

### Pourquoi "écrire un Dockerfile" n'est pas le vrai travail

Écrire un `Dockerfile` de 5 lignes prend 5 minutes. Voici ce qui prend réellement du temps :

| Cas réel                                                              | Pourquoi c'est un problème                                                                                                           | Ce qu'il faut faire                                                                                |
|-----------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------|
| Mosquitto refuse toutes les connexions par défaut                     | Le broker MQTT, par sécurité, bloque les connexions anonymes dès qu'on lui donne un fichier de config                                | Créer un fichier <code>mosquitto.conf</code> explicite qui autorise les connexions (voir plus bas) |
| Le backend démarre avant que la base de données soit prête            | <code>depends_on</code> démarre juste les conteneurs dans l'ordre, mais <b>n'attend pas</b> qu'un service soit vraiment opérationnel | Ajouter un <code>healthcheck</code>                                                                |
| Les données InfluxDB/Grafana disparaissent à chaque redémarrage       | Sans volume, tout ce qui est écrit dans un conteneur est perdu quand il s'arrête                                                     | Utiliser des volumes nommés                                                                        |
| Erreur "port already in use"                                          | Un autre programme (ou un ancien conteneur) utilise déjà le port                                                                     | Savoir diagnostiquer avec <code>docker ps</code> , changer le port, ou arrêter l'ancien conteneur  |
| Le conteneur backend ne peut pas écrire dans un dossier monté         | Problème de permissions entre l'utilisateur du conteneur et celui de ta machine                                                      | Comprendre les utilisateurs Linux dans un conteneur                                                |
| L'image fait 1.2 Go alors que ton code fait 50 Ko                     | Mauvais choix d'image de base, dépendances inutiles installées                                                                       | Utiliser une image "slim", nettoyer le cache pip                                                   |
| Un secret (mot de passe InfluxDB) est visible dans le code sur GitHub | Erreur classique de débutant, risque de sécurité réel                                                                                | Utiliser un fichier <code>.env</code> exclu du dépôt ( <code>.gitignore</code> )                   |

---

## Semaine 1 — Ce qui se cache derrière ton premier Dockerfile

### Jour 1 — Comprendre le choix de l'image de base

```
dockerfile
```

```
FROM python:3.11-slim
```

■ Pourquoi `slim` et pas juste `python:3.11` : l'image standard fait ~1 Go (elle contient plein d'outils dont tu n'as pas besoin), la version `slim` fait ~150 Mo. Pour un projet où plusieurs images tournent en même temps (backend, Mosquitto, InfluxDB, Grafana), la taille compte vraiment — sur ta machine et sur celles d'Ameni/Nour.

**Ce que tu apprends concrètement** : compare toi-même la taille des deux images :

```
bash
```

```
docker images
```

### Jour 2 — L'ordre des instructions compte (mise en cache)

```
dockerfile
```

```
FROM python:3.11-slim
```

```
WORKDIR /app
```

```
# Copier requirements.txt AVANT le reste du code
```

```
COPY requirements.txt .
```

```
RUN pip install --no-cache-dir -r requirements.txt
```

```
# Copier le code APRÈS
```

```
COPY . .
```

■ **Cache de build Docker** : Docker réutilise les étapes précédentes si rien n'a changé. Si tu copies TOUT le code avant d'installer les dépendances, **chaque modification de ton code force à réinstaller toutes les bibliothèques** — ça peut prendre plusieurs minutes à chaque fois au lieu de quelques secondes.

**Ce que tu apprends concrètement** : modifie une ligne dans `app.py`, relance `docker build`, observe que seule la dernière étape se refait (regarde les lignes "CACHED" dans le terminal).

### Jour 3 — Sécuriser tes secrets (mot de passe, clé API)

Crée `.env` (jamais commité) :

```
INFLUX_PASSWORD=un_mot_de_passe_solide
ANTHROPIC_API_KEY=ta_cle_api
```

Crée `.gitignore`:

```
.env
venv/
__pycache__/
rag_db/
```

Dans `docker-compose.yml`:

```
yaml

services:
  backend:
    build: ./backend
    env_file:
      - .env
```

**Ce que tu apprends concrètement :** vérifie avec `git status` que `.env` n'apparaît jamais dans les fichiers à commiter. Erreur classique de débutant à éviter absolument : pousser une clé API sur GitHub (même dans un dépôt privé — beaucoup de fuites viennent de dépôts rendus publics après coup).

---

## Semaine 2 — Ce qui se cache derrière Mosquitto

### Le piège classique : Mosquitto refuse tout par défaut

Si tu lances juste :

```
yaml

mosquitto:
  image: eclipse-mosquitto:2
  ports:
    - "1883:1883"
```

**Tu vas avoir une erreur** `Connection Refused`. Ce n'est pas un bug, c'est volontaire : Mosquitto refuse les connexions anonymes par sécurité dès qu'il n'a pas de configuration explicite.

### Jour 1 — Créer le vrai fichier de configuration

Crée `mosquitto/config/mosquitto.conf`:

```
listener 1883
allow_anonymous true
persistence true
persistence_location /mosquitto/data/
log_dest file /mosquitto/log/mosquitto.log
```

⚠ `allow_anonymous true` : acceptable pour ton projet de stage (réseau local, pas exposé sur Internet), mais à **ne jamais faire en production réelle** — bon réflexe à mentionner dans ton rapport pour montrer que tu comprends l'enjeu de sécurité.

Mets à jour `docker-compose.yml` :

```
yaml

mosquitto:
  image: eclipse-mosquitto:2
  ports:
    - "1883:1883"
  volumes:
    - ./mosquitto/config:/mosquitto/config
    - mosquitto_data:/mosquitto/data
    - mosquitto_log:/mosquitto/log
```

## Jour 2 — Comprendre les réseaux Docker (pourquoi `localhost` ne marche pas entre conteneurs)

■ **Réseau Docker (bridge network)** : par défaut, Docker Compose crée un réseau privé où chaque service peut joindre les autres **par leur nom** (ex : `mosquitto`, `influxdb`), jamais par `localhost` — `localhost`, à l'intérieur d'un conteneur, désigne le conteneur lui-même, pas la machine hôte ni les autres conteneurs.

**Erreur classique que tu vas rencontrer** : ton script Python dans le conteneur `backend` essaie de se connecter à `localhost:1883` → échec. Il faut utiliser `mosquitto:1883`.

**Ce que tu apprends concrètement** : teste volontairement l'erreur (mets `localhost` au lieu de `mosquitto`), observe le message d'erreur exact, comprends pourquoi.

## Jour 3 — Debugger une connexion qui ne marche pas

```
bash

# Rentrer DANS un conteneur en cours d'exécution pour investiguer
docker exec -it plateforme-iot-backend-1 sh

# Une fois dedans, tester la connexion réseau
ping mosquitto
```

**Ce que tu apprends concrètement :** `docker exec -it <nom> sh` est ta commande de debug la plus utilisée dans tout ton stage — retiens-la, tu vas t'en servir chaque semaine.

---

## Semaine 3 — Ce qui se cache derrière InfluxDB et Grafana

### Jour 1 — Les volumes : comprendre pourquoi tes données disparaissent

```
yaml

influxdb:
  image: influxdb:2.7
  volumes:
    - influx_data:/var/lib/influxdb2    # volume NOMMÉ, persiste après arrêt

volumes:
  influx_data:    # déclaré ici, à la racine du fichier
```

 **Volume nommé vs pas de volume :** sans le `volumes:`, toutes les données InfluxDB sont stockées **dans** le conteneur. Dès que tu fais `docker-compose down`, tout disparaît. Avec un volume nommé, les données survivent même si tu supprimes et recrées le conteneur.

**Ce que tu apprends concrètement :** teste volontairement sans volume d'abord — écris des données, fais `docker-compose down` puis `docker-compose up`, constate qu'elles ont disparu. Puis ajoute le volume et refais le test — les données restent.

### Jour 2 — `depends_on` ne fait pas ce que tu crois

```
yaml

grafana:
  depends_on:
    - influxdb
```

 **Piège :** `depends_on` attend juste qu'InfluxDB **démarre**, pas qu'il soit **prêt à recevoir des requêtes** (InfluxDB peut prendre quelques secondes à s'initialiser complètement). Résultat possible : Grafana démarre et essaie de se connecter avant qu'InfluxDB soit vraiment opérationnel → erreur au premier lancement.

**Solution simple pour ton niveau (pas besoin de healthcheck complexe) :** si ça arrive, relance juste Grafana :

```
bash

docker-compose restart grafana
```

**Ce que tu apprends concrètement :** comprendre la différence entre "conteneur démarré" et "service prêt" — un vrai concept DevOps que tu retrouveras partout en entreprise.

**Jour 3 — Provisionner Grafana automatiquement (pour que ça marche pour toute l'équipe, pas juste chez toi)**

Au lieu de configurer Grafana à la main dans l'interface (ce qui ne serait valable que sur ta machine), déclare tout en fichiers :

```
yaml

# grafana/provisioning/datasources/influxdb.yml
apiVersion: 1
datasources:
  - name: InfluxDB
    type: influxdb
    url: http://influxdb:8086
    access: proxy
    isDefault: true
```

```
yaml

grafana:
  volumes:
    - ./grafana/provisioning:/etc/grafana/provisioning
```

**Ce que tu apprends concrètement :** c'est ce qu'on appelle "l'infrastructure comme du code" (Infrastructure as Code) — tout est dans des fichiers versionnés sur Git, donc Ameni et Nour ont exactement la même config Grafana que toi automatiquement, sans rien configurer à la main.

**PARTIE 2 — LE RAG (Agent 4)**

**Pourquoi "chunking + ChromaDB + query" n'est pas le vrai travail**

| Cas réel                                                                  | Pourquoi c'est un problème                   | Ce qu'il faut faire                                                                     |
|---------------------------------------------------------------------------|----------------------------------------------|-----------------------------------------------------------------------------------------|
| Le chunk fait 500 caractères mais coupe une phrase importante en deux     | Perte d'information                          | Ajuster <code>chunk_size</code> / <code>chunk_overlap</code> , tester plusieurs valeurs |
| La recherche retourne des passages qui n'ont rien à voir avec la question | Mauvaise qualité de récupération (retrieval) | Ajouter des métadonnées, filtrer par type de document                                   |

| Cas réel                                                                  | Pourquoi c'est un problème                | Ce qu'il faut faire                                                     |
|---------------------------------------------------------------------------|-------------------------------------------|-------------------------------------------------------------------------|
| L'assistant invente une réponse alors qu'aucun document ne parle du sujet | Hallucination du LLM                      | Prompt qui force à dire "je ne sais pas" si le contexte est insuffisant |
| La base de connaissances grossit et les recherches deviennent floues      | Trop de documents non pertinents mélangés | Organiser par catégories (métadonnées), pas juste tout entasser         |
| Comment savoir si le RAG "marche vraiment bien" ?                         | Sans mesure, tu avances à l'aveugle       | Construire un petit jeu de test question/réponse                        |

## Semaine 5-6 — Ce qui se cache derrière ton Agent 4

### Jour 1 — Le chunking n'est pas une taille magique

```
python
from langchain_text_splitters import RecursiveCharacterTextSplitter

# Teste PLUSIEURS configurations, pas une seule au hasard
splitter_petit = RecursiveCharacterTextSplitter(chunk_size=200, chunk_overlap=20)
splitter_moyen = RecursiveCharacterTextSplitter(chunk_size=500, chunk_overlap=50)
splitter_grand = RecursiveCharacterTextSplitter(chunk_size=1000, chunk_overlap=100)
```

■ **Pourquoi tester plusieurs tailles :** un chunk trop petit perd le contexte (ex : une erreur et sa solution se retrouvent dans 2 chunks séparés, donc jamais récupérés ensemble). Un chunk trop grand dilue la pertinence (le texte utile se noie dans du texte inutile, la recherche devient moins précise).

**Ce que tu apprends concrètement :** indexe le même document avec les 3 tailles, pose la même question aux 3 versions, compare quelle taille donne la meilleure réponse.

### Jour 2 — Nettoyer les documents avant de les indexer

Un vrai fichier Markdown a du bruit :

```
markdown
# Titre du document
Contenu du document

```

## # Erreur de build

```
<!-- généré automatiquement, ne pas éditer -->
[![build](badge.svg)](lien)
```

## ## Description

L'erreur `'undefined reference'` apparaît...

python

```
import re
```

```
def nettoyer_markdown(texte: str) -> str:
    texte = re.sub(r'<!--.*?-->', '', texte, flags=re.DOTALL) # retire les com
    texte = re.sub(r'\[![.*?\]\(.*?\)\]\(.*?\)', '', texte) # retire les bad
    return texte.strip()
```

**Ce que tu apprends concrètement :** un document "brut" indexé tel quel pollue tes résultats de recherche avec du bruit (badges, liens de navigation, mentions automatiques) — le nettoyage avant indexation est une vraie étape, pas un détail.

## Jour 3 — Ajouter des métadonnées pour filtrer la recherche

python

```
collection.add(
    documents=chunks,
    ids=ids,
    metadatas=[
        {"source": "erreurs_docker", "categorie": "erreur", "date": "2026-07"}
        for _ in chunks
    ]
)

# Recherche filtrée : seulement dans les erreurs Docker, pas tout mélangé
resultats = collection.query(
    query_texts=["port already in use"],
    n_results=3,
    where={"categorie": "erreur"}
)
```

■ **Métadonnées :** des informations attachées à chaque chunk (sa source, sa catégorie, sa date) qui permettent de filtrer la recherche, plutôt que de chercher dans TOUS les documents mélangés à chaque fois.

**Ce que tu apprends concrètement :** sans filtre, une question sur Docker peut ramener un chunk de documentation Zephyr sans rapport, juste parce qu'il contient des mots similaires.



## Jour 4 — Empêcher le LLM d'halluciner

```
python

prompt = f"""
Voici des extraits de documentation :
{chunks_pertinents}

Question : {question}

RÈGLES STRICTES :
- Réponds UNIQUEMENT à partir des extraits ci-dessus.
- Si les extraits ne contiennent pas l'information demandée,
  réponds exactement : "Je n'ai pas trouvé cette information dans la documentat
- Ne complète jamais avec tes connaissances générales.
"""
```

**Ce que tu apprends concrètement :** teste volontairement avec une question hors sujet ("comment cuire des pâtes?") — vérifie que l'agent refuse de répondre au lieu d'inventer un lien avec ton projet. C'est ce qui distingue un RAG fiable d'un simple chatbot qui bluffe.

## Jour 5 — Construire ton propre jeu de test (évaluation)

```
python

jeu_de_test = [
    {
        "question": "L'erreur 'west: command not found' apparaît, que faire ?",
        "reponse_attendue_contient": "pip install west"
    },
    {
        "question": "Pourquoi Mosquitto refuse ma connexion ?",
        "reponse_attendue_contient": "allow_anonymous"
    },
]

for cas in jeu_de_test:
    reponse = demander_a_assistant(cas["question"])
    reussi = cas["reponse_attendue_contient"].lower() in reponse.lower()
    print(f"{'✅' if reussi else '❌'} {cas['question']}")
```

 **Pourquoi c'est important :** sans ça, tu ne sais pas objectivement si ton RAG s'améliore ou empire quand tu changes un paramètre (taille de chunk, prompt, nombre de résultats). Avec ce jeu de test, tu peux comparer objectivement "avant/après" chaque modification.

**Ce que tu apprends concrètement :** c'est une vraie pratique professionnelle (évaluation systématique), pas juste "tester à l'œil" en posant 2-3 questions au hasard.

---

**Ce que ça donne comme vision d'ensemble de tes 5-6 semaines**

| Semaine | Ce que tu VOIS dans le planning | Ce que tu FAIS réellement                                                                                         |
|---------|---------------------------------|-------------------------------------------------------------------------------------------------------------------|
| 1-3     | "Docker complet"                | Debug de connexions réseau, gestion de volumes, sécurisation de secrets, configuration Mosquitto                  |
| 4       | "Agent 1"                       | Gestion d'erreurs, faux positifs/négatifs, décision d'appeler ou non le LLM, tests sur dépôts réels variés        |
| 5-6     | "Agent 4 (RAG)"                 | Nettoyage de documents, tuning du chunking, filtrage par métadonnées, anti-hallucination, évaluation systématique |

**Ce tableau EST ton vrai planning de travail** — beaucoup plus que les titres génériques du cahier des charges.

---

*Ce document complète ton guide principal. Combine-le avec ton backlog ultra-détaillé pour l'ordre exact des tâches.*